git简介

git的诞生

1991年 Linus开创了开源的Linux

2002年以前 Linus手动合并代码

2002年 BitKeeper免费授权

2005年 BitKeeper被尝试破解，收回授权

2005年 Linus花了两周时间用C写出了git，并且一个月之内将Linux搬迁到git上

2008年 GitHub上线

集中式与分布式

集中式

CVS、SVN

分布式

Git

Git安装

git最新版本，没有安装包的话，去官方地址下载https://github.com/git-for-windows/git/releases/download/v2.23.0.windows.1/Git-2.23.0-64-bit.exe

一路next直到安装完毕，保持默认的选项

安装完毕后，打开windows的`cmd`命令界面，输入如下命令，来设置自己的用户名和邮箱。

```
git config --global user.name "Aaron"
git config --global user.email "Aaron@eagleslab.com"
```

创建版本库

版本库又名仓库，英文名**repository**，你可以简单的解成一个目录，这个目录里面的所有文件都可以被Git管理起来，每个文件的修改、删除，Git都能跟踪，以便任何时刻都可以追踪历史，或者在将来某个时刻可以“还原”。

第一步 找个空的地方创建一个文件夹

第二步 在文件夹中打开`cmd`界面，然后输入`git init`命令把这个目录变成Git可以管理的仓库

```
C:\Users\Aaron\Desktop\gitlearn>git init
Initialized empty Git repository in C:/Users/Aaron/Desktop/gitlearn/.git/
```

我们也可以发现，在这个文件夹中会创建一个`.git`的目录， 这个目录是Git来跟踪管理版本库的

添加文件到版本库

这边要注意，git只能追踪文本文件的改动，而二进制文件只能记录大小的变化。

第一步

- 在我们的git文件夹中创建一个记事本文件，不过注意，千万别用windows自带的记事本编辑内容！
- 在记事本文件中写一些内容，比如`学前沿IT，到英格科技`
- 使用命令`git add`告诉git，把文件添加到暂存区中

```
git add readme.txt
```

执行上面的命令，没有任何显示

第二步

用命令`git commit`告诉Git，把文件提交到仓库

```
C:\Users\Aaron\Desktop\gitlearn>git commit -m "write a readme file"
[master (root-commit) 4121352] write a readme file
 1 file changed, 1 insertions(+)
 create mode 100644 readme.txt
```

`git commit`后面的`-m`是输入本次提交的说明的，可以输入任何内容，最好是自己能看懂的，这样就可以从理事会记录里面方便的找到改动记录。

执行成功后`1 file changed` 一个文件被改动了，也就是我们添加的`readme.txt`

`1 insertions(+)`表示插入了一行内容，我们写的那个`学前沿IT，到英格科技`

注意

因为在`commit`前需要`add`一下，所以可以一次提交多个文件

```
C:\Users\Aaron\Desktop\gitlearn>git add file1.txt
C:\Users\Aaron\Desktop\gitlearn>git add file2.txt
C:\Users\Aaron\Desktop\gitlearn>git commit -m "two files"
[master dc43809] two files
 2 files changed, 2 insertions(+)
 create mode 100644 file1.txt
 create mode 100644 file2.txt
```

回滚操作

我们修改上面的文件`readme.txt`

然后使用`git status`查看

```
C:\Users\Aaron\Desktop\gitlearn>git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git checkout -- <file>..." to discard changes in working directory)

        modified:   readme.txt

no changes added to commit (use "git add" and/or "git commit -a")
```

`git status`命令可以让我们时刻掌握仓库当前的状态，上面的命令输出告诉我们，`readme.txt`被修改过了，但还没有准备提交的修改。

如果想看看具体修改了什么内容，我们可以使用`git diff`来查看

```
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/gitlearn$ git diff readme.txt
diff --git a/readme.txt b/readme.txt
index 28f9c19..3fc856b 100644
--- a/readme.txt
+++ b/readme.txt
@@ -1 +1,2 @@
-学前沿IT，到英格科技！
\ No newline at end of file
+学前沿IT，到英格科技！^M
+网络，系统，信息安全，云计算，开发全覆盖！
\ No newline at end of file
```

可以看到在第一行后面加了一个换行符，然后加了一行在下面

下面我们将其添加，然后查看`status`

```
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/gitlearn$ git add readme.txt
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/gitlearn$ git status readme.txt
On branch master
Changes to be committed:
  (use "git reset HEAD <file>..." to unstage)

        modified:   readme.txt
```

`git status`告诉我们，将要被提交的修改包括`readme.txt`，下一步，就可以放心地提交了

```
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/gitlearn$ git commit -m "add distributed"
[master 597b614] add distributed
 1 file changed, 1 insertion(+), 1 deletion(-)
```

提交后，我们再用`git status`命令看看仓库的当前状态：

```
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/gitlearn$ git status
On branch master
nothing to commit, working tree clean
```

Git告诉我们当前没有需要提交的修改，而且，工作目录是干净（working tree clean）的

版本回退

我们创建一个文件叫`game`，在这个文件中写入以下内容

```
屠龙勇士村来了一个新勇士叫"林克"
```

然后提交，提交的说明就是"新的开始"

```
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/gitlearn$ nano game
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/gitlearn$ git add game
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/gitlearn$ git commit -m "新的开始"
[master 0512417] 新的开始
 1 file changed, 1 insertion(+)
 create mode 100644 game
```

在文件中追加内容

```
勇士赤手空拳来到了野外
```

然后提交，提交的说明就是"走出新手村"

```
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/gitlearn$ nano game
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/gitlearn$ git add game
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/gitlearn$ git commit -m "走出新手村"
[master 4cf7a68] 走出新手村
 1 file changed, 1 insertion(+)
```

在文件中追加内容

```
勇士林克被Lv1怪物史莱姆一屁股坐死
```

然后提交，提交的说明就是"阵亡"

```
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/gitlearn$ nano game
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/gitlearn$
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/gitlearn$ git add game
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/gitlearn$ git commit -m "阵亡"
[master 04d2c8c] 阵亡
 1 file changed, 1 insertion(+)
```

在文件中追加内容

```
勇士林克被公主"塞尔达"所救，丢失全部金币！
```

然后提交，提交的说明就是"复活"

```
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/gitlearn$ nano game
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/gitlearn$ git add game
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/gitlearn$ git commit -m "复活"
[master b696a63] 复活
 1 file changed, 1 insertion(+)
```

下面我们来看下这个悲催的勇士的经历，可以使用`git log`查看历史提交

```
commit b696a637b97faf113da41207ad0be026084d3ea9 (HEAD -> master)
Author: Aaron <aaron@eagleslab.com>
Date:   Sat Oct 26 21:26:04 2019 +0800

    复活

commit 04d2c8cb5f7cf43956a176538d14c3db2adf0159
Author: Aaron <aaron@eagleslab.com>
Date:   Sat Oct 26 21:24:30 2019 +0800

    阵亡

commit 4cf7a68d96ed57f1818ea7caec68c0697338c80f
Author: Aaron <aaron@eagleslab.com>
Date:   Sat Oct 26 21:22:57 2019 +0800

    走出新手村

commit 05124170e9143b19c33e3edfb74b9ab0f042361d
Author: Aaron <aaron@eagleslab.com>
Date:   Sat Oct 26 21:21:01 2019 +0800

    新的开始
```

`git log`命令显示从最近到最远的提交日志，我们可以看到3次提交

如果嫌输出信息太多，看得眼花缭乱的，可以试试加上`--pretty=oneline`参数

```
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/gitlearn$ git log --pretty=oneline
b696a637b97faf113da41207ad0be026084d3ea9 (HEAD -> master) 复活
04d2c8cb5f7cf43956a176538d14c3db2adf0159 阵亡
4cf7a68d96ed57f1818ea7caec68c0697338c80f 走出新手村
05124170e9143b19c33e3edfb74b9ab0f042361d 新的开始
```

其实勇士的悲剧完全可以逆转，我们可以**读档**到`新的开始`，让勇士在出门前带把武器

```
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/gitlearn$ git reset --hard 05124
HEAD is now at 0512417 新的开始
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/gitlearn$ cat game
屠龙勇士村来了一个新勇士叫"陈松"
```

Git的版本回退速度非常快，因为Git在内部有个指向当前版本的`HEAD`指针，当你回退版本的时候，Git仅仅是把HEAD从指向`新的开始`：

```
┌────┐
│HEAD│
└────┘
   │
   └──> ○ 复活
        │
        ○ 阵亡
        │
        ○ 走出新手村
        │
        ○ 新的开始
```

改为指向`add distributed`：

```
┌────┐
│HEAD│
└────┘
   │
   │    ○ 复活
   │    │
   │    ○ 阵亡
   │    │
   │    ○ 走出新手村
   │    │
   └──>○ 新的开始
```

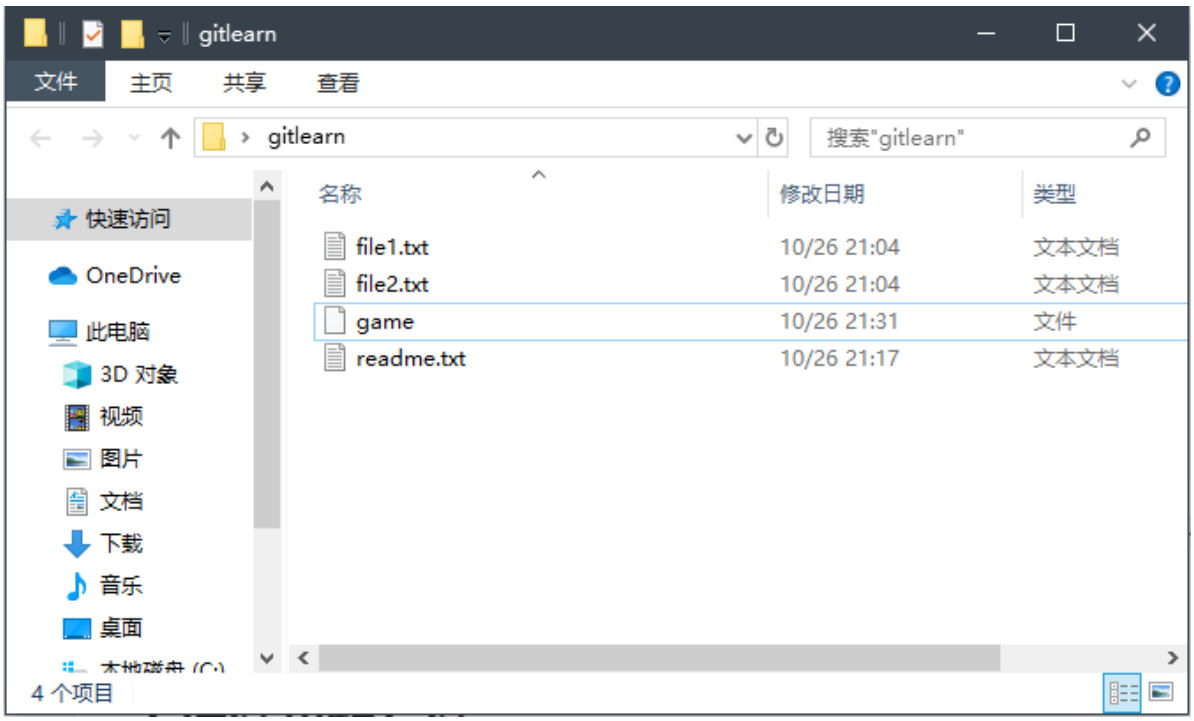
在Git中，用`HEAD`表示当前版本，上一个版本就是`HEAD^`，上上一个版本就是`HEAD^^`，当然往上100个版本写100个`^`比较容易数不过来，所以写成`HEAD~100`。

Git提供了一个命令`git reflog`用来记录你的每一次命令：

工作区和暂存区

工作区

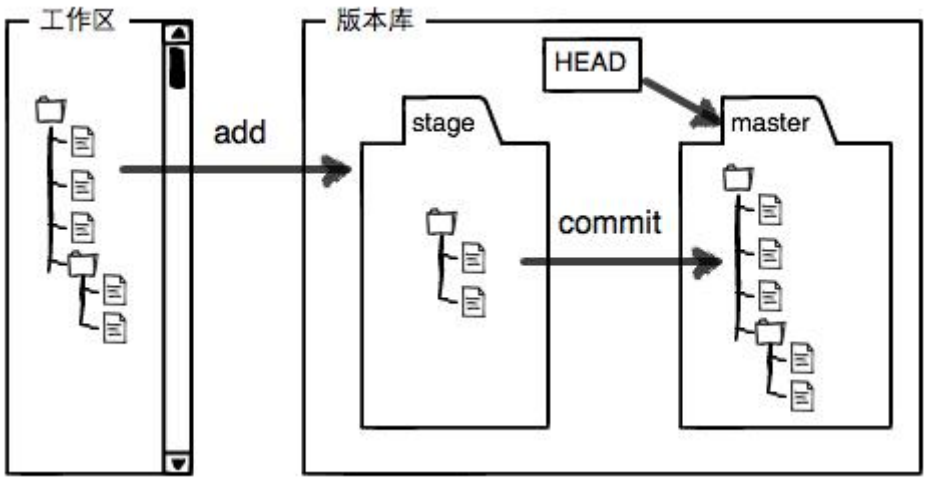
就是你在电脑里能看到的目录，比如上面的`gitlearn`文件夹



版本库

工作区有一个隐藏目录`.git`，这个不算工作区，而是Git的版本库。

Git的版本库里存了很多东西，其中最重要的就是称为stage（或者叫index）的暂存区，还有Git为我们自动创建的第一个分支`master`，以及指向`master`的一个指针叫`HEAD`。



前面讲了我们把文件往Git版本库里添加的时候，是分两步执行的：

第一步是用`git add`把文件添加进去，实际上就是把文件修改添加到暂存区；

第二步是用`git commit`提交更改，实际上就是把暂存区的所有内容提交到当前分支。

因为我们创建Git版本库时，Git自动为我们创建了唯一一个`master`分支，所以，现在，`git commit`就是往`master`分支上提交更改。

你可以简单理解为，需要提交的文件修改通通放到暂存区，然后，一次性提交暂存区的所有修改。

管理修改

git不是管理文件，而是管理修改的操作。

我们修改readme.txt，然后add提交到暂存区

```
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/gitlearn$ cat readme.txt
学前沿IT，到英格科技！
网络，系统，信息安全，云计算，开发全覆盖!!
修改一哈
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/gitlearn$ git add readme.txt
```

然后再次修改readme.txt，这次我们直接commit

```
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/gitlearn$ cat readme.txt
学前沿IT，到英格科技！
网络，系统，信息安全，云计算，开发全覆盖!!
修改一哈
修改二哈
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/gitlearn$ git commit -m "猜猜是一还是二"
[master c793c64] 猜猜是一还是二
 1 file changed, 1 insertion(+)
```

然后查看状态

```
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/gitlearn$ git status
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git checkout -- <file>..." to discard changes in working directory)

        modified:   readme.txt

no changes added to commit (use "git add" and/or "git commit -a")
```

我们会发现第二次修改的并没有被提交

查看一下工作区的文件和仓库的文件的区别

```
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/gitlearn$ git diff head -- readme.txt
diff --git a/readme.txt b/readme.txt
index bce6d8c..6b08054 100644
--- a/readme.txt
+++ b/readme.txt
@@ -1,3 +1,4 @@
 学前沿IT，到英格科技！
 网络，系统，信息安全，云计算，开发全覆盖!!
 修改一哈
+修改二哈^M
```

撤销修改

- 当你改乱了工作区某个文件的内容，想直接丢弃工作区的修改时，用命令`git checkout -- file`。
- 当你不但改乱了工作区某个文件的内容，还添加到了暂存区时，想丢弃修改，分两步，第一步用命令`git reset HEAD`，就回到了上一种情况，第二步按上一种情况操作操作。

删除文件

我们将文件从工作区删除

```
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/gitlearn$ rm file1.txt
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/gitlearn$ git status
On branch master
Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git checkout -- <file>..." to discard changes in working directory)

        deleted:    file1.txt
        modified:   readme.txt

no changes added to commit (use "git add" and/or "git commit -a")
```

git会直接察觉到我们的删除操作，如果这个时候提交，那么就会从版本库中删除该文件

```
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/gitlearn$ git add .
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/gitlearn$ git commit -m "deleted file1"
[master f5943ff] deleted file1
 2 files changed, 1 insertion(+), 1 deletion(-)
 delete mode 100644 file1.txt
```

但是如果误删除了，可以还原到版本库中的最新版本

```
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/gitlearn$ git reset --hard c793
HEAD is now at c793c64 猜猜是一还是二
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/gitlearn$ ls
file1.txt  file2.txt  game  readme.txt
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/gitlearn$ cat file1.txt
我是file1ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/gitlearn$
```

如果要删除暂存区中的文件，可以使用`git rm`

远程仓库

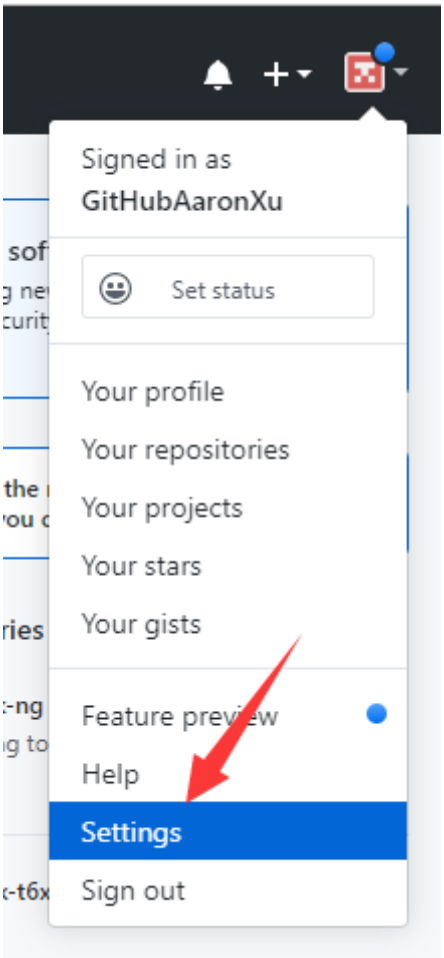
注册github

打开命令行，创建key

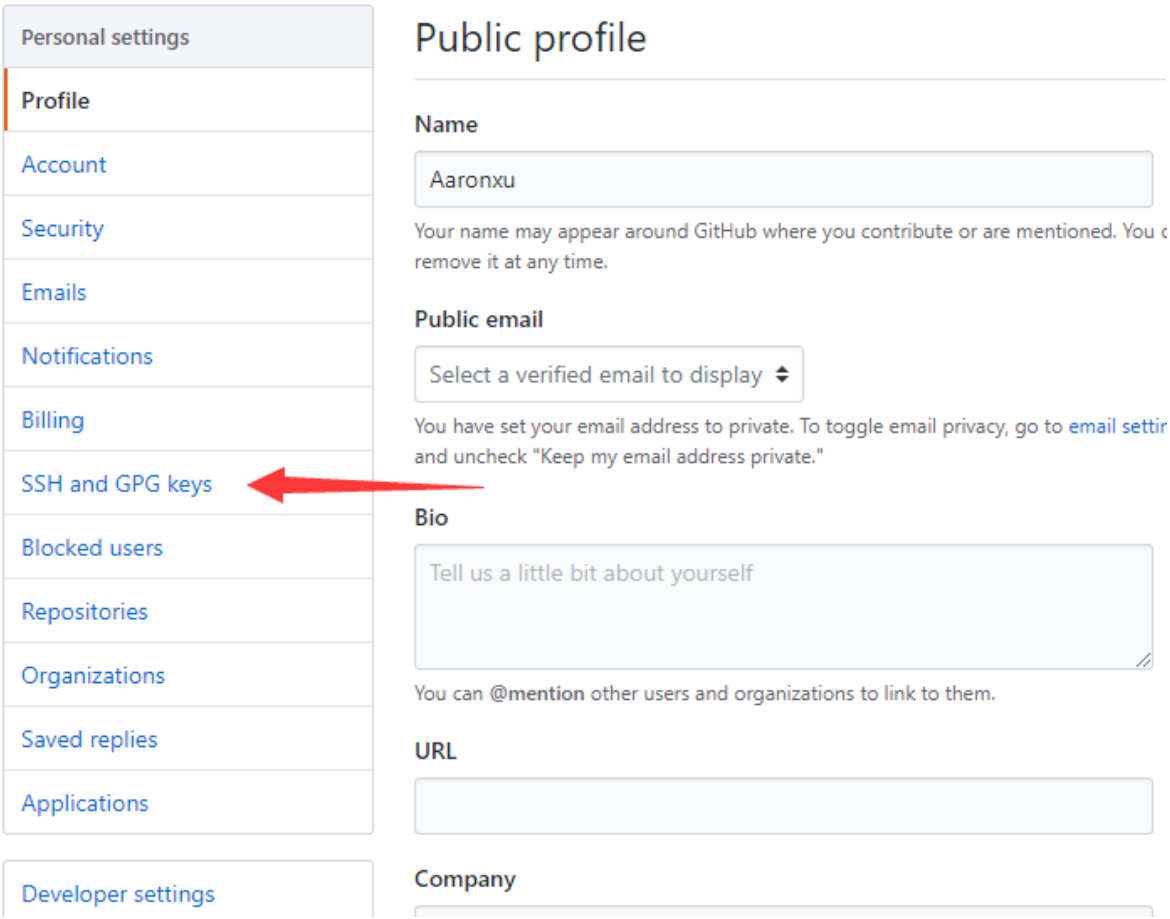
```
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/gitlearn$ ssh-keygen -t rsa -C "aaron@eagleslab.com"
Generating public/private rsa key pair.
Enter file in which to save the key (/home/ubuntu/.ssh/id_rsa):
Created directory '/home/ubuntu/.ssh'.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/ubuntu/.ssh/id_rsa.
Your public key has been saved in /home/ubuntu/.ssh/id_rsa.pub.
The key fingerprint is:
SHA256:LXQFNsmAzGN7zsnnboLVFIrNgnli4i1Zdn9/9geOZjs aaron@eagleslab.com
The key's randomart image is:
+---[RSA 2048]----+
|      o ..o+o.    |
|       *  .+o     |
|      + *....     |
|   . B *.=o.      |
| . B + BS+.       |
|  + .   B.+   .   |
|   .   o + . o .  |
|       . . o E + .|
|          +.o.= .o|
+----[SHA256]-----+
```

注意一下这个文件的保存位置，下面要用

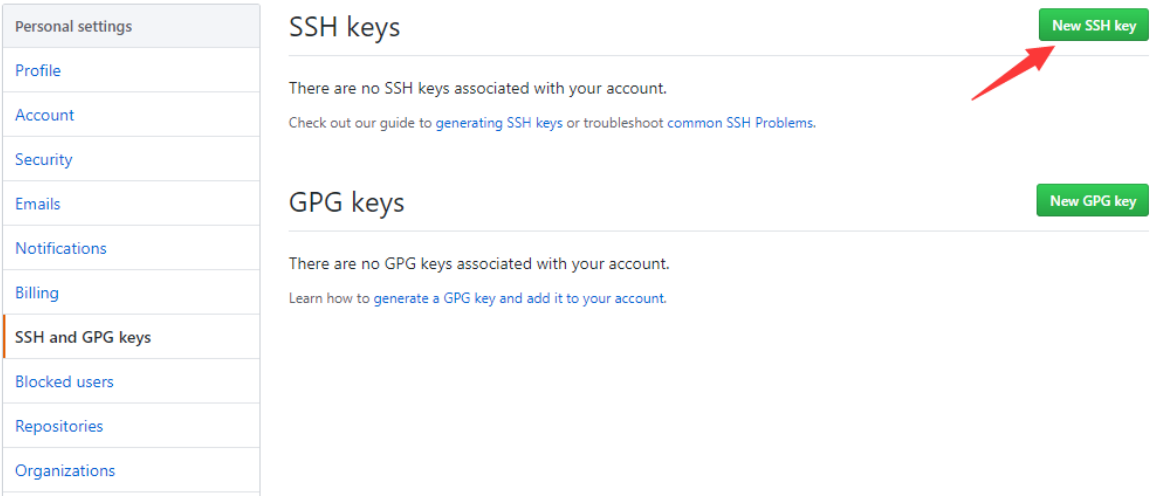
打开github的个人用户设置



在里面找到 `SSH and GPG keys`

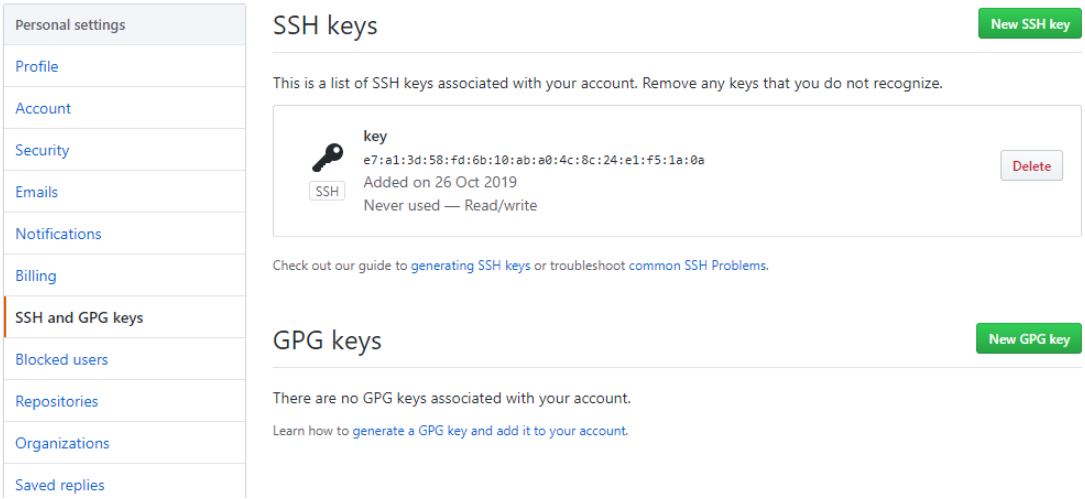


新建一个ssh keys



然后按照下图提示填写

然后你就成功添加了ssh的key，下次登录github的时候，就会直接被识别出你的身份啦

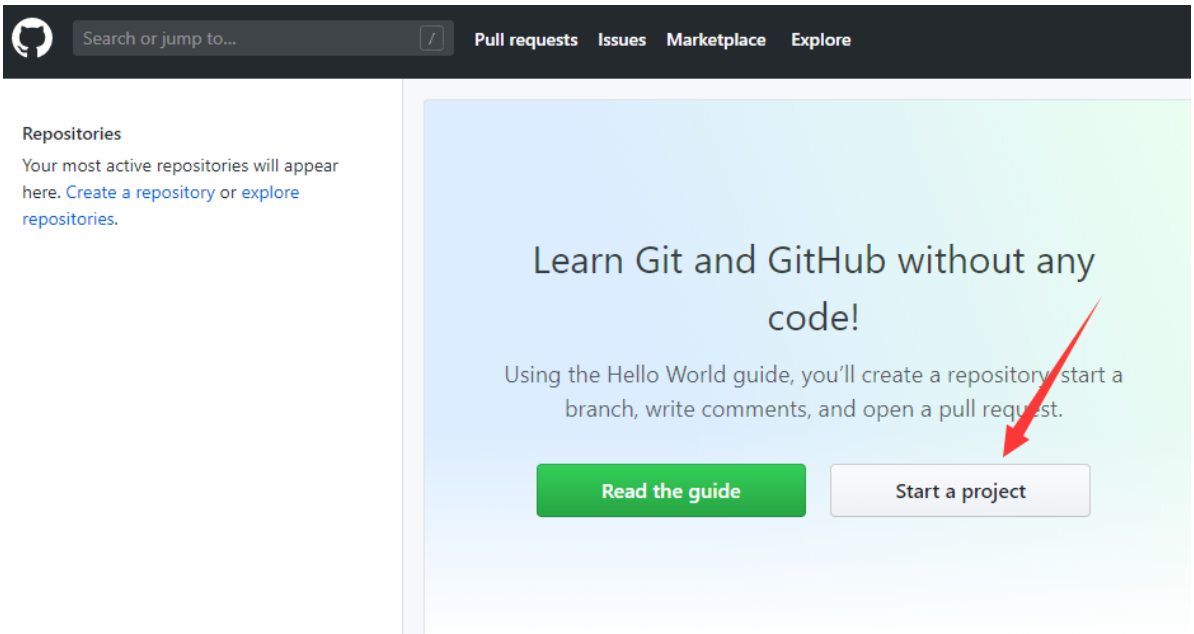


使用如下命令可以测试是否可以成功登录

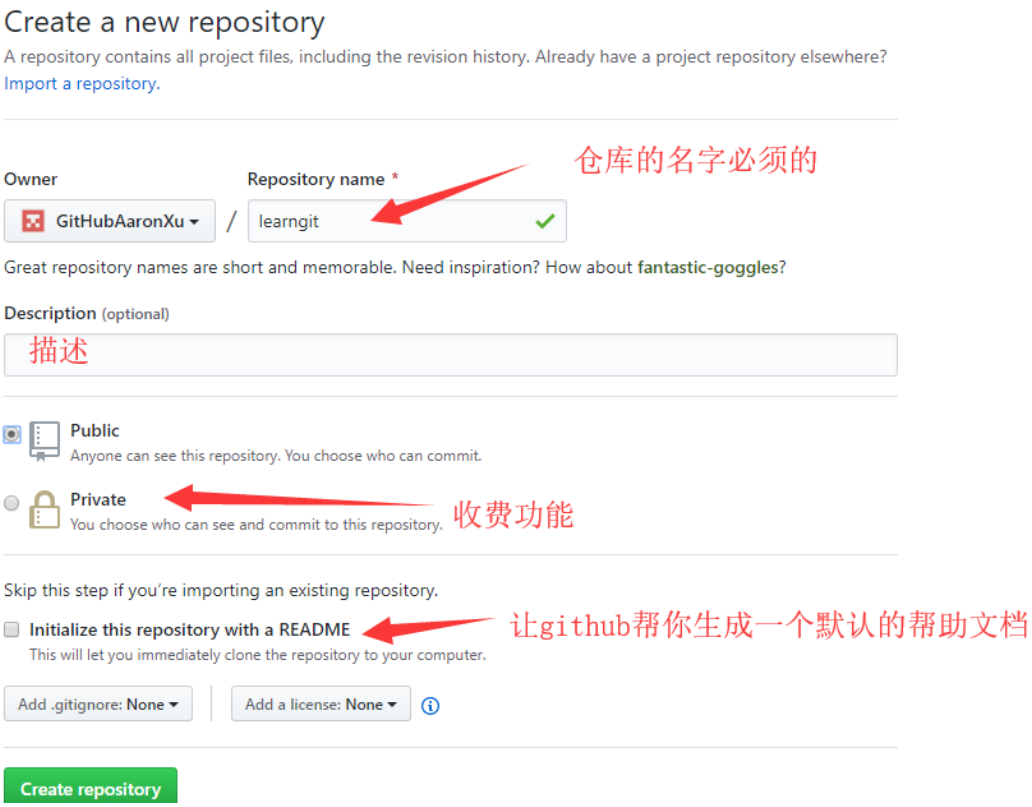
```
ssh git@github.com
```

添加远程仓库

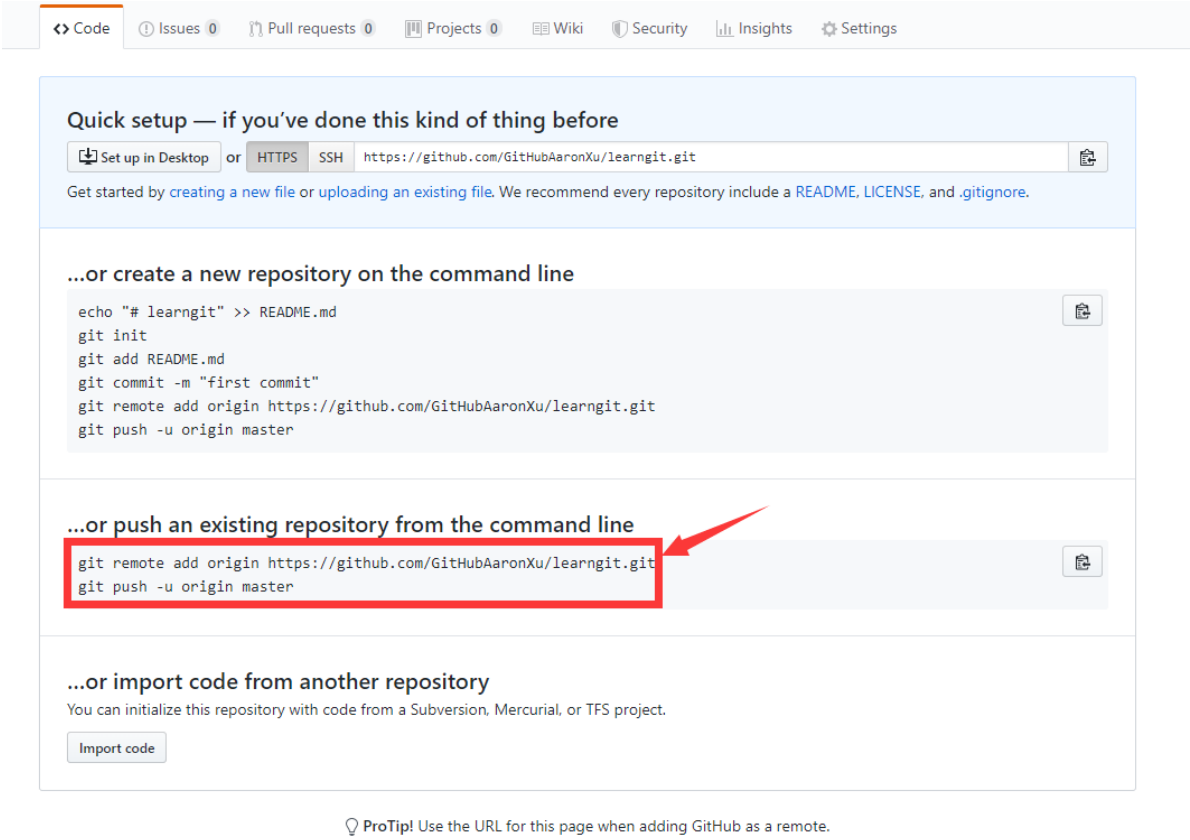
首先回到登录之后的页面



创建你的仓库



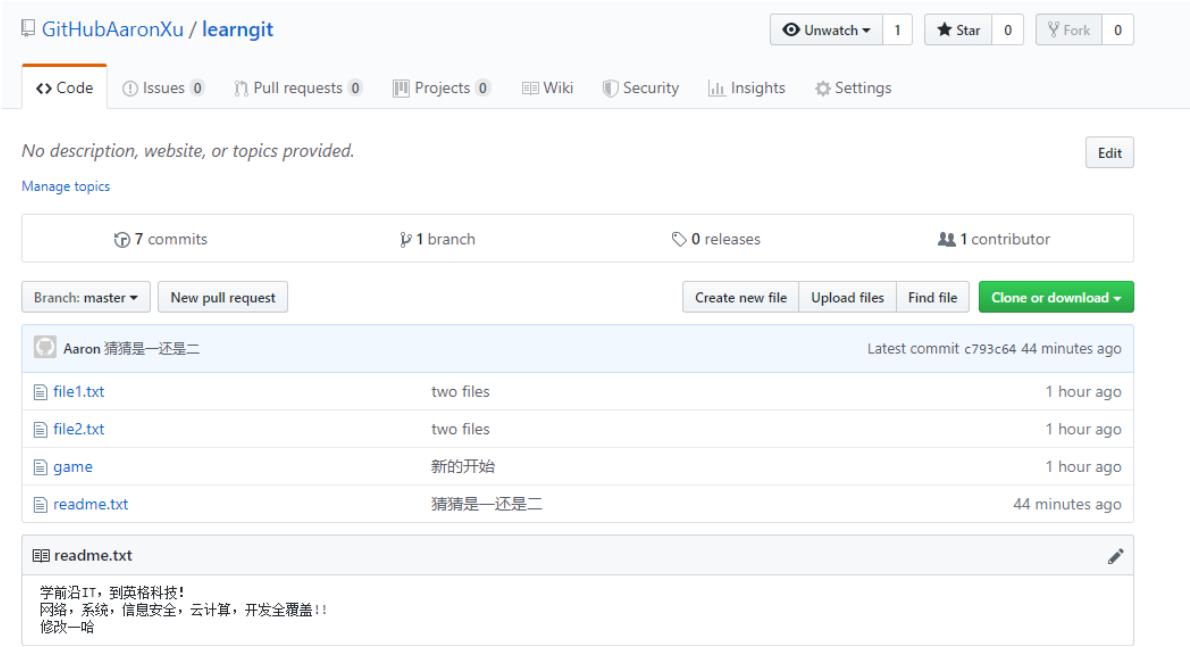
根据这边的提示可以将我们本地的仓库和远程的仓库关联起来哦



推送成功的画面

```
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/gitlearn$ git push -u origin master
Username for 'https://github.com': GitHubAaronXu
Password for 'https://GitHubAaronXu@github.com':
Counting objects: 22, done.
Delta compression using up to 8 threads.
Compressing objects: 100% (15/15), done.
Writing objects: 100% (22/22), 1.75 KiB | 94.00 KiB/s, done.
Total 22 (delta 6), reused 0 (delta 0)
remote: Resolving deltas: 100% (6/6), done.
To https://github.com/GitHubAaronXu/learngit.git
 * [new branch]      master -> master
Branch 'master' set up to track remote branch 'master' from 'origin'.
```

观察到远程仓库和我们本地仓库已经关联一致



要关联一个远程库，使用命令`git remote add origin git@server-name:path/repo-name.git`；

关联后，使用命令`git push -u origin master`第一次推送master分支的所有内容；

此后，每次本地提交后，只要有必要，就可以使用命令`git push origin master`推送最新修改；

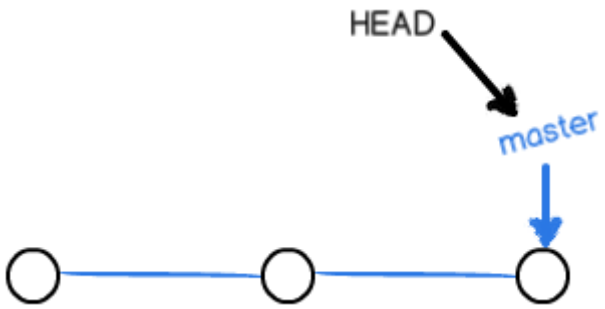
从远程仓库克隆

```
git clone <地址>
```

分支管理

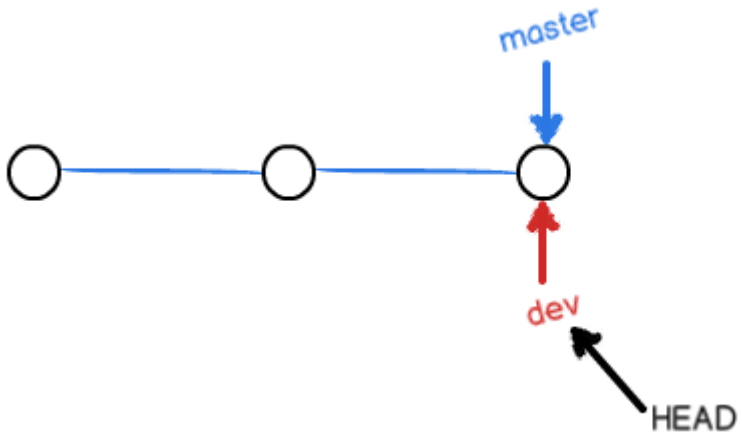
创建与合并分支

一开始的时候，`master`分支是一条线，Git用`master`指向最新的提交，再用`HEAD`指向`master`，就能确定当前分支，以及当前分支的提交点：



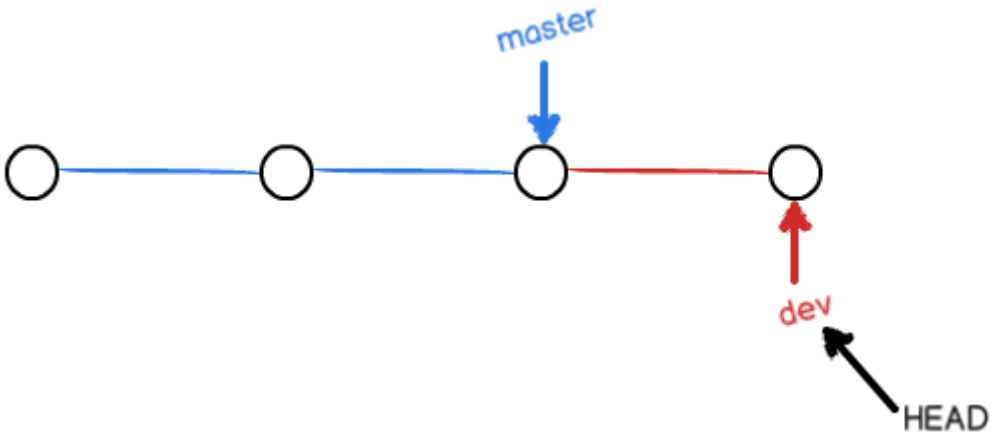
每次提交，`master`分支都会向前移动一步，这样，随着你不断提交，`master`分支的线也越来越长。

当我们创建新的分支，例如`dev`时，Git新建了一个指针叫`dev`，指向`master`相同的提交，再把`HEAD`指向`dev`，就表示当前分支在`dev`上：

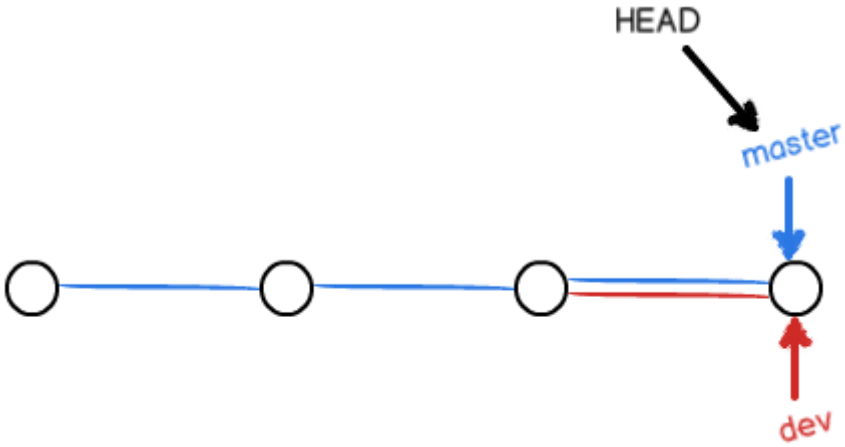


你看，Git创建一个分支很快，因为除了增加一个`dev`指针，改改`HEAD`的指向，工作区的文件都没有任何变化！

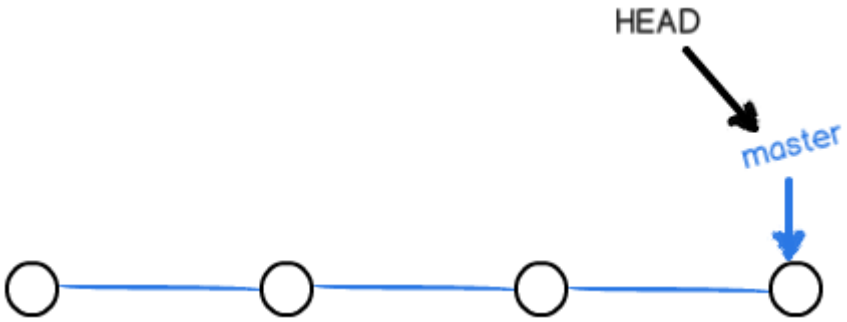
不过，从现在开始，对工作区的修改和提交就是针对`dev`分支了，比如新提交一次后，`dev`指针往前移动一步，而`master`指针不变：



假如我们在`dev`上的工作完成了，就可以把`dev`合并到`master`上。Git怎么合并呢？最简单的方法，就是直接把`master`指向`dev`的当前提交，就完成了合并：



合并完分支后，甚至可以删除`dev`分支。删除`dev`分支就是把`dev`指针给删掉，删掉后，我们就剩下了一条`master`分支：



首先，我们创建`dev`分支，然后切换到`dev`分支：

```
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/learngit$ git checkout -b dev
Switched to a new branch 'dev'
```

`git checkout`命令加上`-b`参数表示创建并切换，相当于以下两条命令：

```
$ git branch dev
$ git checkout dev
Switched to branch 'dev'
```

然后，用`git branch`命令查看当前分支：

```
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/learngit$ git branch
* dev
  master
```

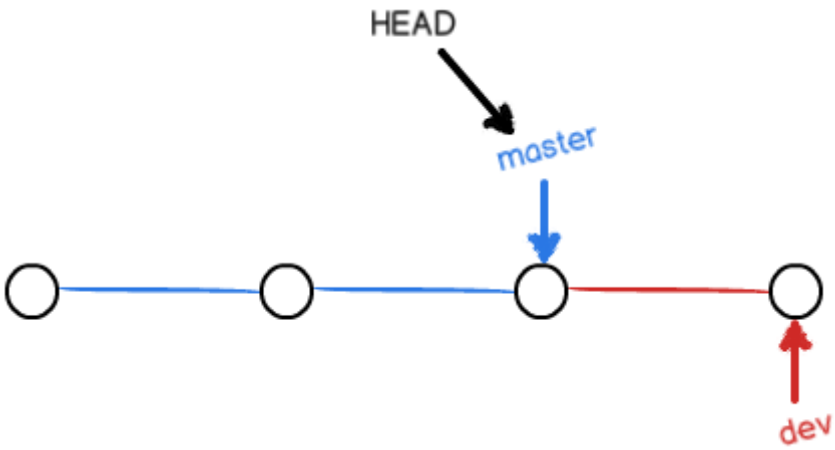
`git branch`命令会列出所有分支，当前分支前面会标一个`*`号。

然后，我们就可以在`dev`分支上正常提交，比如对`readme.txt`做个修改，加上一行。

然后提交

```
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/learngit$ git add readme.txt
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/learngit$ git commit -m "branch readme"
[dev 5fdc15d] branch readme
 1 file changed, 2 insertions(+), 1 deletion(-)
```

切换回`master`分支后，再查看一个`readme.txt`文件，刚才添加的内容不见了！因为那个提交是在`dev`分支上，而`master`分支此刻的提交点并没有变



现在，我们把`dev`分支的工作成果合并到`master`分支上：

```
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/learngit$ git checkout master
Switched to branch 'master'
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/learngit$ git merge dev
Updating 5467fb7..5fdc15d
Fast-forward
 readme.txt | 3 ++-
 1 file changed, 2 insertions(+), 1 deletion(-)
```

`git merge`命令用于合并指定分支到当前分支。合并后，再查看`readme.txt`的内容，就可以看到，和`dev`分支的最新提交是完全一样的。

注意到上面的`Fast-forward`信息，Git告诉我们，这次合并是“快进模式”，也就是直接把`master`指向`dev`的当前提交，所以合并速度非常快。

合并完成后，就可以放心地删除`dev`分支了：

```
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/learngit$ git branch -d dev
Deleted branch dev (was 5fdc15d).
```

删除后，查看`branch`，就只剩下`master`分支了：

```
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/learngit$ git branch
* master
```

switch

最新版本的Git提供了新的`git switch`命令来切换分支

创建并切换到新的`dev`分支，可以使用：

```
$ git switch -c dev
```

直接切换到已有的`master`分支，可以使用：

```
$ git switch master
```

使用新的`git switch`命令，比`git checkout`要更容易理解。

命令总结

Git鼓励大量使用分支：

查看分支：`git branch`

创建分支：`git branch`

切换分支：`git checkout`或者`git switch`

创建+切换分支：`git checkout -b`或者`git switch -c`

合并某分支到当前分支：`git merge`

删除分支：`git branch -d`

解决冲突

准备新的`feature1`分支，继续我们的新分支开发：

```
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/learngit$ git checkout -b feature1
Switched to a new branch 'feature1'
```

修改`readme.txt`最后一行

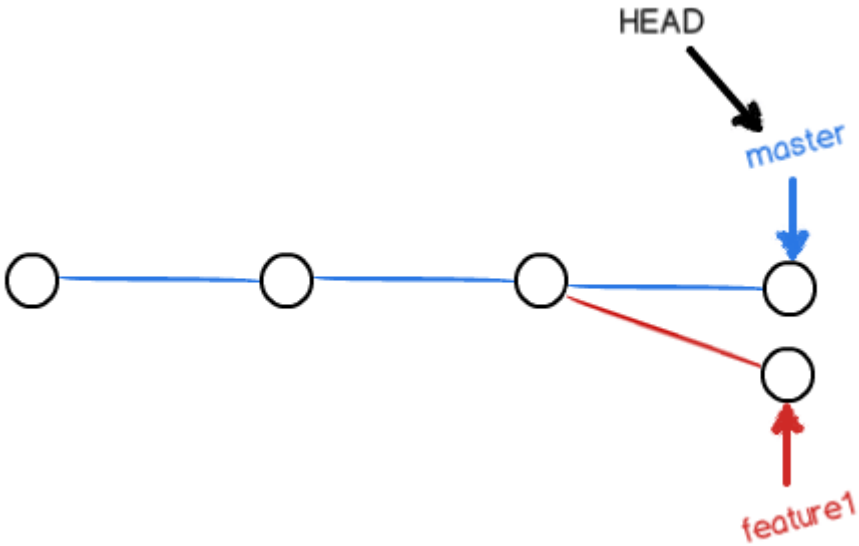
在`feature1`分支上提交

```
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/learngit$ git add readme.txt
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/learngit$ git commit -m "add feature1"
[feature1 a3cab77] add feature1
 1 file changed, 1 insertion(+)
```

切换到`master`分支，修改`readme.txt`最后一行为别的内容，然后提交

```
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/learngit$ git add readme.txt
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/learngit$ git commit -m "add master"
[master dfffa4b] add master
 1 file changed, 1 insertion(+)
```

现在，`master`分支和`feature1`分支各自都分别有新的提交，变成了这样：



Git无法执行“快速合并”，只能试图把各自的修改合并起来，但这种合并就可能会有冲突

```
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/learngit$ git merge feature1
Auto-merging readme.txt
CONFLICT (content): Merge conflict in readme.txt
Automatic merge failed; fix conflicts and then commit the result.
```

Git告诉我们，`readme.txt`文件存在冲突，必须手动解决冲突后再提交。`git status`也可以告诉我们冲突的文件：

```
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/learngit$ git status
On branch master
You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Unmerged paths:
  (use "git add <file>..." to mark resolution)

        both modified:   readme.txt

no changes added to commit (use "git add" and/or "git commit -a")
```

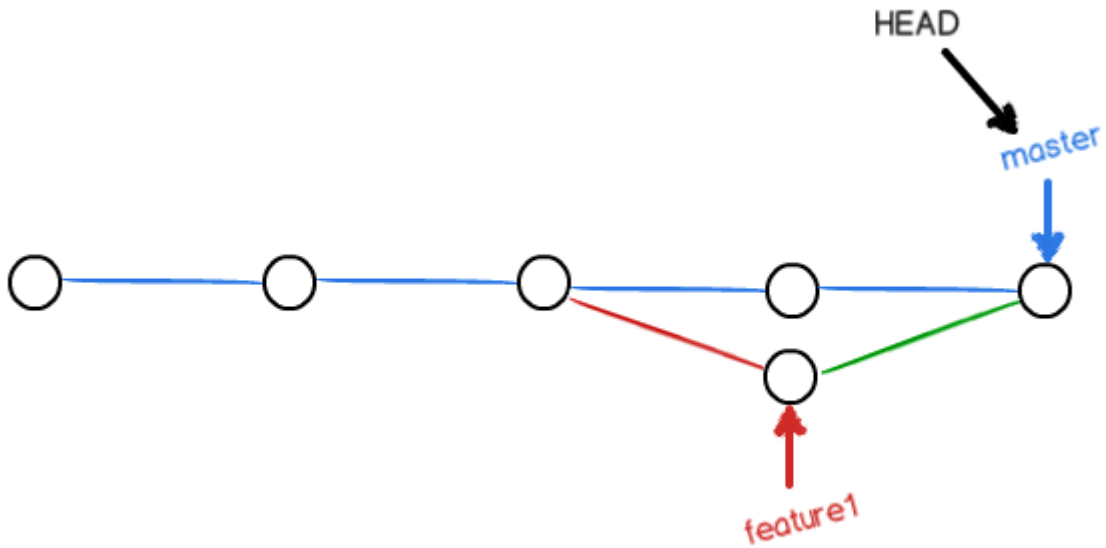
我们可以直接查看readme.txt的内容：

```
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/learngit$ cat readme.txt
hello world
bug
bug
bug
new line
<<<<<<< HEAD
master
=======
feature1
>>>>>>> feature1
```

Git用`<<<<<<<`，`=======`，`>>>>>>>`标记出不同分支的内容，我们修改后保存，然后提交：

```
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/learngit$ git add readme.txt
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/learngit$ git commit -m "conflict fixed"
[master ea16955] conflict fixed
```

现在，`master`分支和`feature1`分支变成了下图所示：



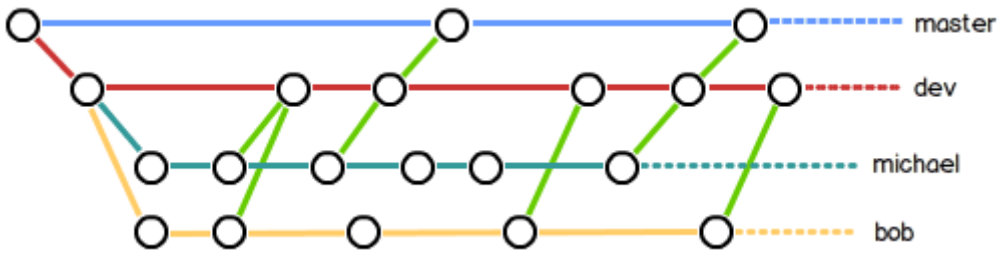
用带参数的`git log`也可以看到分支的合并情况：

```
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/learngit$ git log --graph --pretty=oneline --abbrev-commit
*   ea16955 (HEAD -> master) conflict fixed
|\
| * a3cab77 (feature1) add feature1
* | dfffa4b add master
|/
* 5fdc15d branch readme
* 5467fb7 新的存档
* 0c54d1b 有写了一个bug
* 02ea45c 提交一个bug
* 4ab716a 创建一个readme
```

分支策略

首先，`master`分支应该是非常稳定的，也就是仅用来发布新版本，平时不能在上面干活

所以，团队合作的分支看起来就像这样



标签管理

创建标签

首先，切换到需要打标签的分支上：

```
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/learngit$ git branch
  feature1
* master
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/learngit$ git checkout feature1
Switched to branch 'feature1'
```

然后，敲命令`git tag`就可以打一个新标签：

```
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/learngit$ git tag v1.0
```

可以用命令`git tag`查看所有标签：

```
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/learngit$ git tag
v1.0
```

可以给历史的commit id加上标签

```
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/learngit$ git log --pretty=oneline --abbrev-commit
a3cab77 (HEAD -> feature1, tag: v1.0) add feature1
5fdc15d branch readme
5467fb7 新的存档
0c54d1b 有写了一个bug
02ea45c 提交一个bug
4ab716a 创建一个readme
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/learngit$ git tag v0.9 5fdc
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/learngit$ git tag
v0.9
v1.0
```

注意，标签不是按时间顺序列出，而是按字母排序的。可以用`git show`查看标签信息：

```
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/learngit$ git show v0.9
commit 5fdc15d42929266573c2fdb1d59d37961dcccb56 (tag: v0.9)
Author: Aaron <aaron@eagleslab.com>
Date:   Sun Oct 27 12:47:35 2019 +0800

    branch readme

diff --git a/readme.txt b/readme.txt
index 833abb1..da17aeb 100644
--- a/readme.txt
+++ b/readme.txt
@@ -1,4 +1,5 @@
 hello world
 bug
 bug
-bug
\ No newline at end of file
+bug^M
+new line^M
```

还可以创建带有说明的标签，用`-a`指定标签名，`-m`指定说明文字：

```
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/learngit$ git tag -a v0.1 -m "version 0.1 released" 4ab716
```

用命令`git show`可以看到说明文字：

```
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/learngit$ git show v0.1
tag v0.1
Tagger: Aaron <aaron@eagleslab.com>
Date:   Sun Oct 27 13:19:55 2019 +0800

version 0.1 released

commit 4ab716a493db1064ab6e504e3afb075ca30844b4 (tag: v0.1)
Author: Aaron <aaron@eagleslab.com>
Date:   Sun Oct 27 10:43:46 2019 +0800

    创建一个readme

diff --git a/readme.txt b/readme.txt
new file mode 100644
index 0000000..95d09f2
--- /dev/null
+++ b/readme.txt
@@ -0,0 +1 @@
+hello world
\ No newline at end of file
```

操作标签

如果标签打错了，也可以删除：

```
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/learngit$ git tag -d v0.1
Deleted tag 'v0.1' (was 8b80044)
```

因为创建的标签都只存储在本地，不会自动推送到远程。所以，打错的标签可以在本地安全删除。

如果要推送某个标签到远程，使用命令`git push origin`：

```
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/learngit$ git push origin v1.0
```

如果标签已经推送到远程，要删除远程标签就麻烦一点，先从本地删除：

```
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/learngit$ git tag -d v0.9
Deleted tag 'v0.9' (was 5fdc15d)
```

然后，从远程删除。删除命令也是push，但是格式如下：

```
ubuntu@DESKTOP-HQ0R9B5:/mnt/c/Users/Aaron/Desktop/learngit$ git push origin :refs/tags/v0.9
```

- 命令`git push origin`可以推送一个本地标签；
- 命令`git push origin --tags`可以推送全部未推送过的本地标签；
- 命令`git tag -d`可以删除一个本地标签；
- 命令`git push origin :refs/tags/`可以删除一个远程标签。